

Unconventional Initial Access Vectors Pt. 1



seccore.at/blog/itsecx2025

Benjamin Floriani, Patrick Pongratz

October 6, 2025



The ITSecX conference [1](#) has been a yearly must-visit for both of us. Meeting former colleagues and long-standing friends from the community and listening to excellent talks is always a great experience we never want to miss! So we were especially happy that, for the first time, we would be presenting as our own company and taking part in the conference instead of just attending.

Preparation

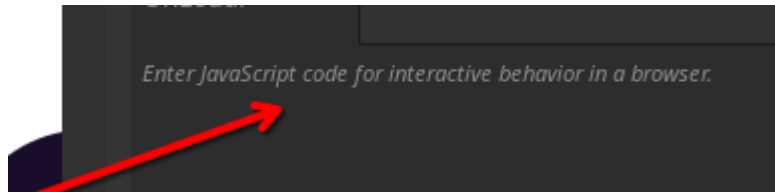
We knew that we wanted to do a talk about something realistic, that ties in with our passion for Red Teaming and offensive security. Choosing the track (there are usually multiple simultaneous tracks at ITSecX, like defence, offense, embedded etc.) was easy for us, but finding a suitable topic was not that trivial. We did not want to recycle something that we already did, or just do a "fun" talk with no technical research behind it. So we decided it had to be something related to [Red Team Engagements](#), ideally a topic that is not widely discussed yet and also offers some value to our listeners on how to protect against it.

A New Idea

While preparing for the talk, there was also a lot of other work involved when founding a company. Apart from printing and signing lots of paperwork, there were also enjoyable tasks, such as designing and refining a logo to represent us. We used Inkscape² for this task, which is an open source vector graphic editor perfect for this. It saves logos as

Scalable Vector Graphics (SVG) files that are widely used and are usually opened in a web-browser when double-clicked.

Just for fun, we also played with the logo and tried to animate it. This can be done via JavaScript and that's when the idea for our talk was born: SVG as an initial access vector.

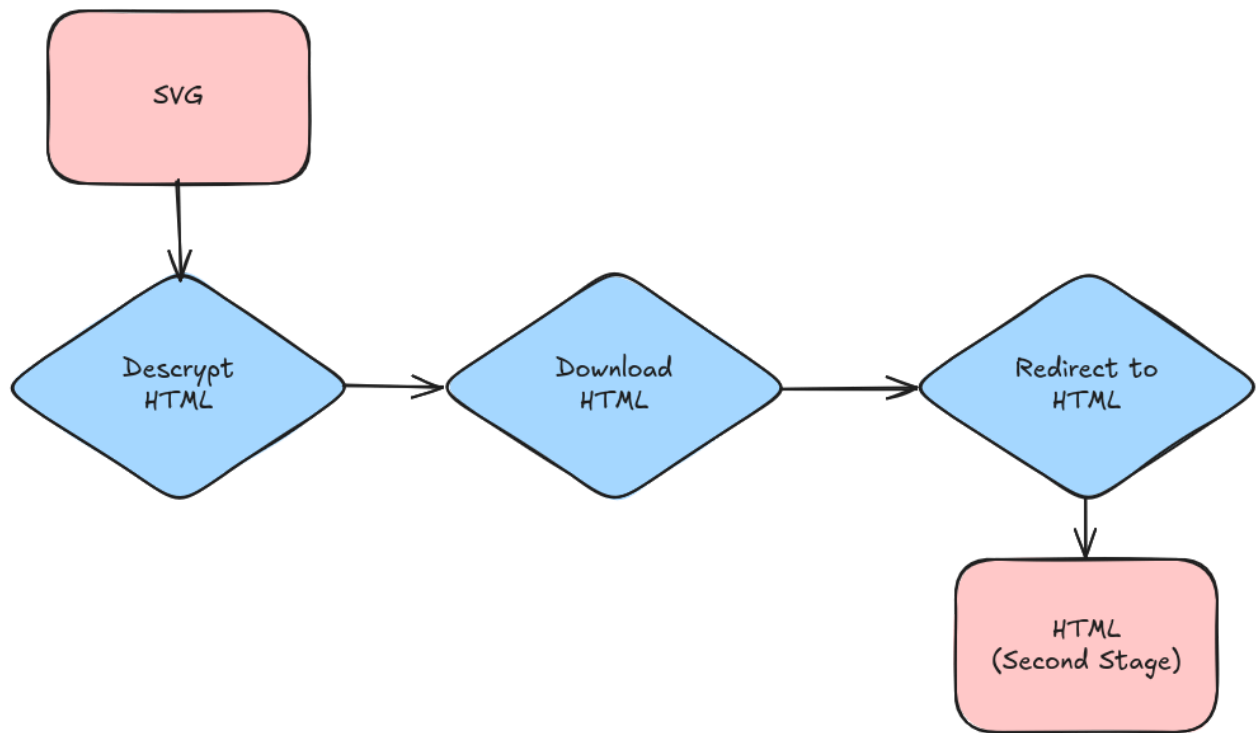


Initial Research

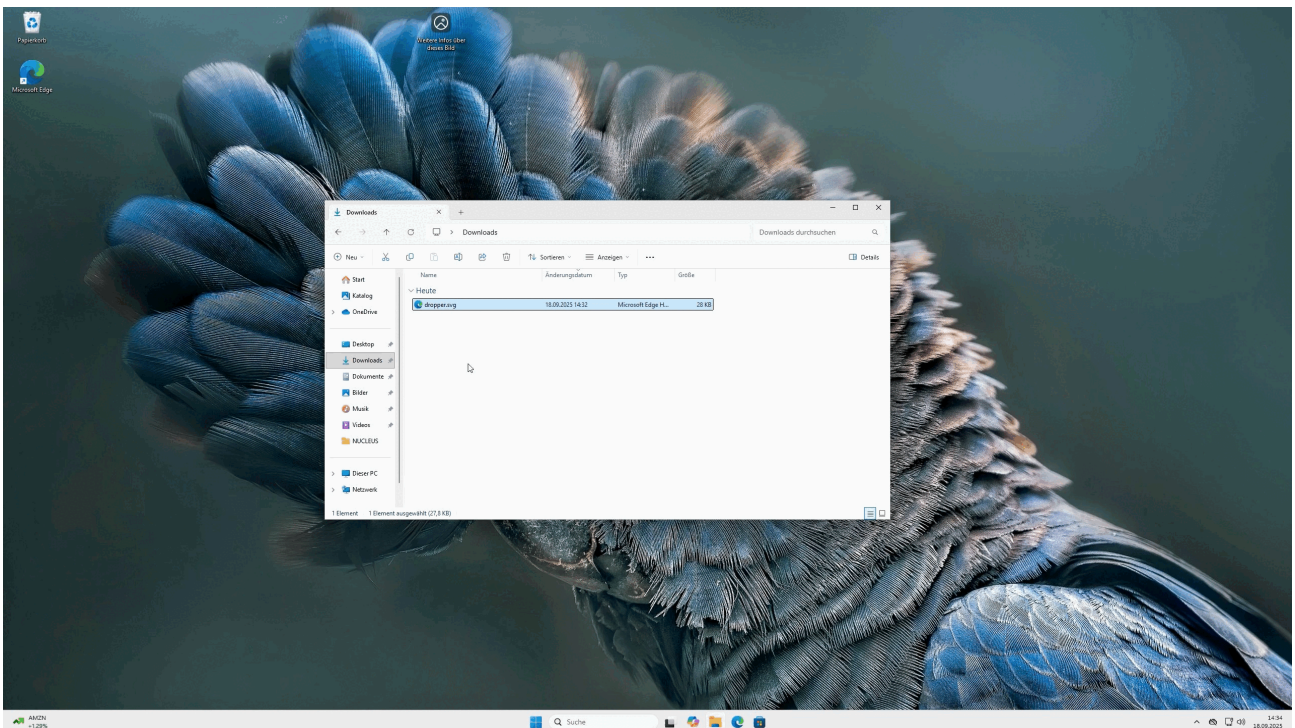
The idea of using local JavaScript for Red Team Engagements is not new. We both have used HTML files with embedded JavaScript before, however, HTML files normally get blocked by mail gateways and users usually do not receive legitimate HTML files as part of their daily business. This makes them easily detectable by most security teams. SVG files on the other hand are commonly used, especially in design departments. Legitimate emails can also embed or attach SVG files as part of their corporate design and signature, making them harder to generally block.

However, there was a problem we needed to overcome: SVG files are not HTML files, so they do not have a Document Object Model (DOM) that can be used to create a local-only phishing page. Instead of trying to recreate one using pure SVG design, we settled for a different approach: Create a two-stage payload that drops an HTML file into the Downloads folder, and redirect to that file. We used the same JavaScript download technique that is commonly used to download payloads from a local HTML file:

- Encrypt, encode and embed the file inside the SVG using a JavaScript variable
- Decrypt the file using JavaScript
- Create a hidden link element
- Click that hidden link element
- Wait for the download to finish and redirect to the downloaded file



The process of opening the SVG file is represented in this gif:

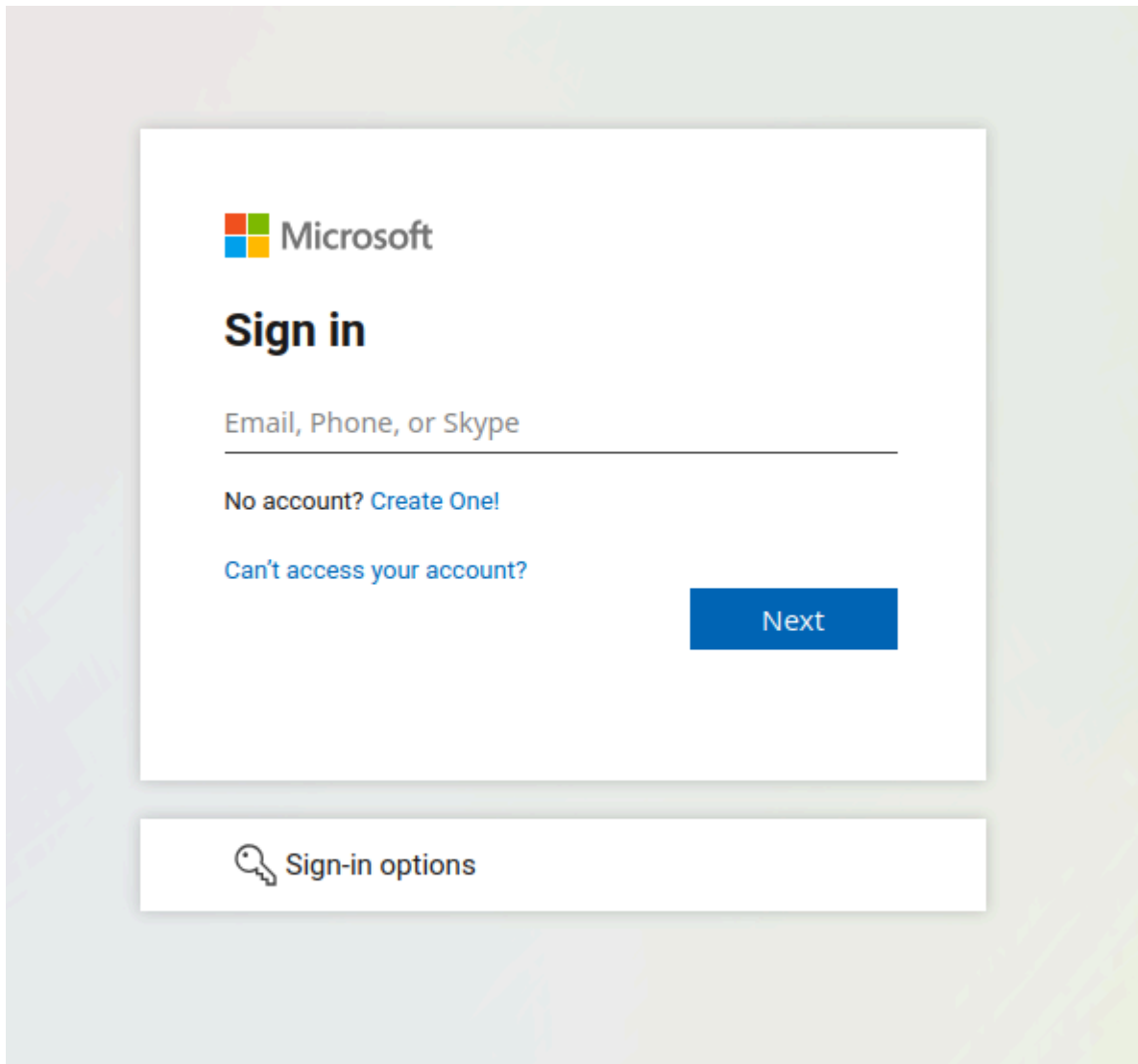


This approach gave us a lot of flexibility, since everything happens in the local JavaScript engine. Each step of our payload can contain sandbox evasion techniques and stop the execution if a detection is triggered. The payload is not even going to be decrypted, if a sandbox is detected, giving us more possibilities to stay undetected.

The Second Stage

Everything from now on is technically not about SVG as an initial access vector anymore. However, we wanted to include a full attack chain to show the real-life impact such an attack can have.

The payload that gets dropped via our SVG is a classic HTML phishing page. For this talk, we chose a Microsoft Entra-ID login page, since it is the most common identity management platform in Austrian enterprises. The HTML file is then automatically opened in the browser and a user is prompted to input their credentials:



When credentials are entered, they are sent to our backend.

The Backend

Up until now, no network or internet connection was established, neither by the SVG nor the HTML file. But in order to receive the credentials and pass on the Multi-Factor Authentication (MFA) number that is used in Microsoft Authenticator MFA, one request needs to be done:



Approve sign in request



Open your Authenticator app and approve the request. Enter the number if prompted.

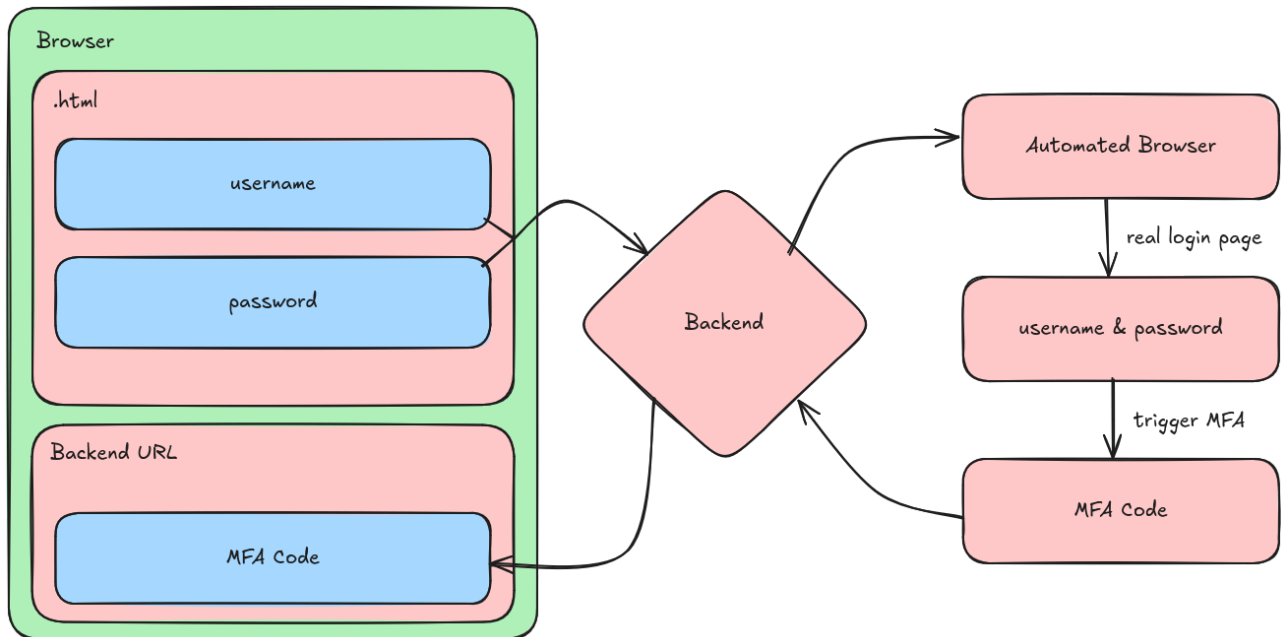
31

Didn't receive a sign-in request? **Swipe down to refresh** the content in your app.

[I can't use my Microsoft Authenticator app right now](#)

[More information](#)

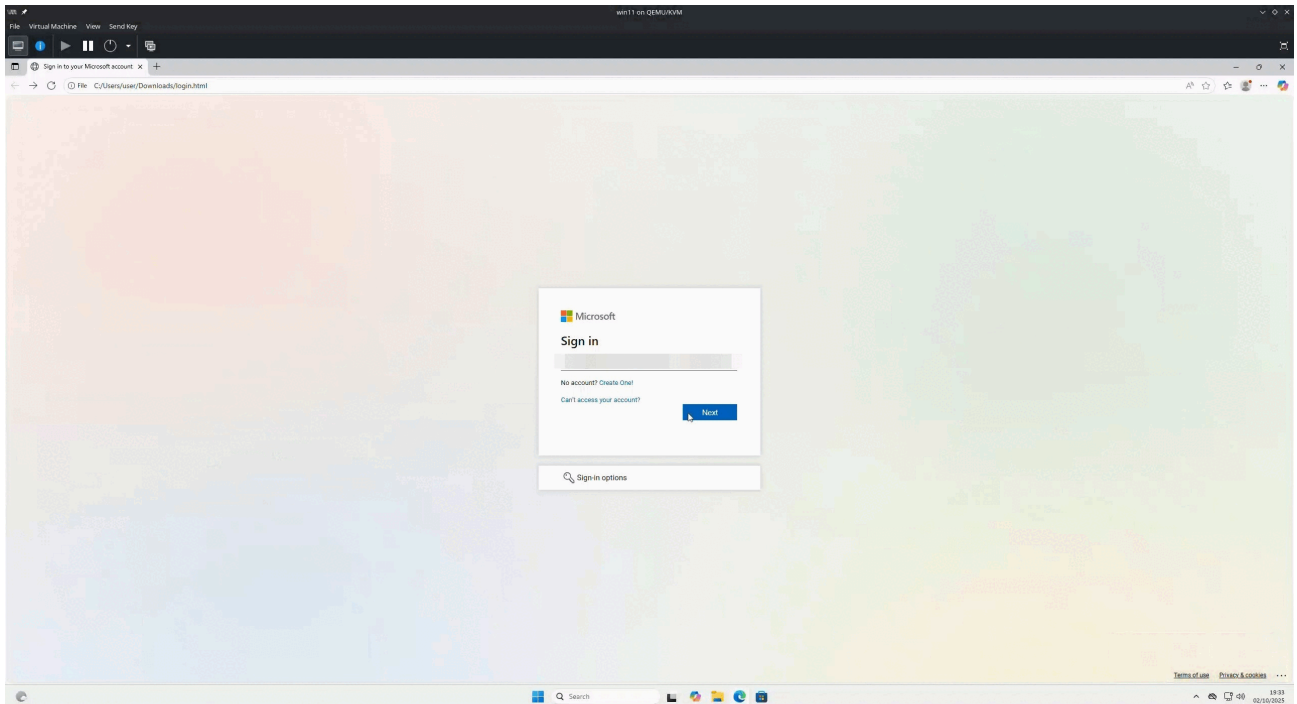
Explaining our backend in depth is out of scope for this initial access vector, we will maybe later dedicate a whole post about our setup. We will just skip to the most relevant parts. Our backend is a very portable piece of software that simply receives credentials over the internet. Then, depending on the engagement and setup, various MFA challenges can be triggered, by using the received credentials to log in with a fully automated browser. If the credentials are correct, a MFA challenge pops up and (in this case the Microsoft Authenticator number) gets returned to the phished browser. Once the sign-in is approved by the user, the automated browser is now fully authenticated and the backend automatically extracts tokens from it. In real engagements, we now use this browser to gather information and run further attacks.



Even though this is not indefinitely scalable, which we do not need in Red Team Engagements, it has several advantages for us:

- The backend itself can be hosted anywhere: As a container, serverless function and even on an ESP32
- It is very adaptable and adding a new MFA challenge or login flow can be easily done
- The automated browser can be run separately from the backend. This gives us several interesting scenarios:
 - We can run the backend and browser in the same region as our target, bypassing geoblocking
 - If we get our hands on a low privileged account and an Entra-joined device, we can run the browser on that device to bypass device restrictions and spear-phish a more sensitive account

The whole process is demonstrated here:



Wrapping Up

We hope this research raises awareness of the dangers of opening any file format in a browser by default. Any environment in which scripts can be executed can be dangerous, even if they are run in a sandbox, as is the case with a web browser.

Mitigation

There are several ways to protect your company from SVG based phishing attacks:

- Block SVG files on the mail-gateway, if they are not used
- Inspect SVG files for embedded scripts
- Change the default file association to SVG files from a browser to a viewer without JavaScript support or a text editor

Want to see our research and how your company reacts to real-world attacks in action?

[Contact us for a free initial consultation!](#)

Recording of our Talk

You can find the recording of our talk on YouTube:



[Watch on YouTube](#)